UNIVERSITY OF TEXAS ARLINGTON | COLLEGE OF BUSINESS

Temporal Prediction of Weather (Temperature)



GaiaDhi

Capstone Project Report: Gaiadhi

Course: INSY-5379-001-Business Analytics Capstone

Supervisor: Prof. Dr. Jayarajan Samuel

Team-3
Abhishek Basnet
Jayaram Amgoth
Naveenkumar Sellappan
Sushmitha Ashok Ugle
Venkat Sai Reddy Tipparthi
Yuvarekha Mahendran

Acknowledgement

We would like to express our gratitude to all our group members and others who helped us directly and indirectly to conduct this project work. We express our heart full indebtedness and deep sense of gratitude to instructor **Prof. Dr. Jayarajan Samuel**, for offering us the opportunity to conduct this practical project and his sincere guidance and inspiration to accomplish this project.

We are also thankful to **Mr. Karthik Kannappan** and **Mr. Vasu Vijay** for their time-to-time guidance and their valuable time to make the project understandable to us.

Furthermore, we acknowledge the use of various online platforms and data repositories that provided essential datasets and documentation related to Machine Learning algorithms. We utilized resources from the NASA POWER API, NOAA(Ground station) python, Excel, which were instrumental in the development and execution of this project.

Table of Contents

Introduction 5

About GaiaDhi 5

Objectives 6

Primary Objective 6

Secondary Objective 6

Challenges 6

Data Availability & Quality 6

Methodology Overview 6

Dataset Overview 8

Sources 8

Data Fusion 8

Exploratory Data Analysis (EDA) 9

Correlation Analysis 10

Pattern Analysis 10

Data Preprocessing 13

Handling Missing Values 13

Imputation 14

Model Building 14

Approach 14

Linear Regression 15

Support Vector Regression (SVR) 17

ARIMA (Auto Regressive Integrated Moving Average) 18

SARIMA (Seasonal Auto Regressive Integrated Moving Average) 21

LSTM (Long Short-Term Memory) 23

Random Forest Regressor 25

XG Boost Regressor 29

Performance Evaluation **32**

Prediction For Other Station **33**

Limitations **34**

Future Recommendations **34**

AI-driven Weather Forecasting Models **35**

References **37**

Introduction

Accurate weather prediction plays a vital role in various aspects of modern life, particularly in agriculture, disaster preparedness, and resource management. For farmers, weather is not just a background detail—it's a critical factor that influences irrigation planning, pest and disease control, and crop yield optimization. Even minor changes in temperature or unexpected rainfall can significantly affect productivity and livelihoods. At the same time, precise weather forecasting is essential for disaster management teams, who depend on early warnings to prepare for extreme events like floods, hurricanes, heatwaves, and droughts. Timely and reliable predictions can save lives, safeguard infrastructure, and help minimize economic losses across vulnerable regions.

Traditionally, weather forecasts have been powered by Numerical Weather Prediction (NWP) models. These models use complex mathematical equations to simulate atmospheric behavior based on physical laws. While they have served as the foundation of forecasting for decades, they come with limitations-particularly in terms of localized accuracy, high computational demands, and real-time adaptability.

Recent advances in Artificial Intelligence (AI) and Machine Learning (ML) have opened new possibilities for weather prediction. Models like Google's GraphCast have shown that deep learning can rival or even outperform traditional forecasting methods in both accuracy and speed. These AI models learn from vast historical weather data to identify complex patterns often missed by conventional approaches. One of the biggest advantages of ML is its ability to generate highly localized forecasts. This is especially useful in agriculture, where weather conditions can vary significantly even over short distances. As a result, AI-driven forecasts enable better planning, risk reduction, and more efficient usage of resources.

This project explores the potential of AI and ML to enhance weather forecast specifically focusing on predicting temperature using ground station data. The goal is to improve forecast accuracy at a local level and evaluate the practicality of integrating machine learning models. The focus is on improving the accuracy and localization of weather predictions.

About GaiaDhi

In the evolving agricultural landscape, where productivity and profitability are paramount, the integration of cutting-edge technologies has become essential. GaiaDhi, a trailblazer in agricultural innovation, has harnessed the potential of machine learning and crop science through its flagship platform, Holos—a cloud-based Crop Modeling as a Service (CMaaS) platform. Holos goes beyond traditional data analysis by incorporating crop simulation models (CSMs), enabling informed decision-making for sustainable and profitable farming. By leveraging advanced analytics, crop science, and modular solutions, GaiaDhi has positioned itself as a catalyst for empowering the farming and research communities.

This project, in collaboration with GaiaDhi, aims to develop an ML-based weather prediction model that leverages local ground station data to forecast temperature. By evaluating the efficacy of short-term (7-day to 14 day) predictions, this study will assess the feasibility of integrating AI-driven weather forecasting solutions for improved decision-making in agriculture and beyond.

Objectives

Primary Objective

- Develop a Machine Learning (ML) model that predicts maximum and minimum temperature using local ground station data.
- Evaluate the temporal prediction efficacy for short-term (7-day) and extended (14-day) forecasts.

Secondary Objective

- Analyze the feasibility of AI-driven weather forecasting models, such as Google GraphCast, Meteomatics, and IBM Environmental Intelligence Suite.
- Provide a detailed comparison of these models based on integration feasibility, tenure, cost, and computational requirements.

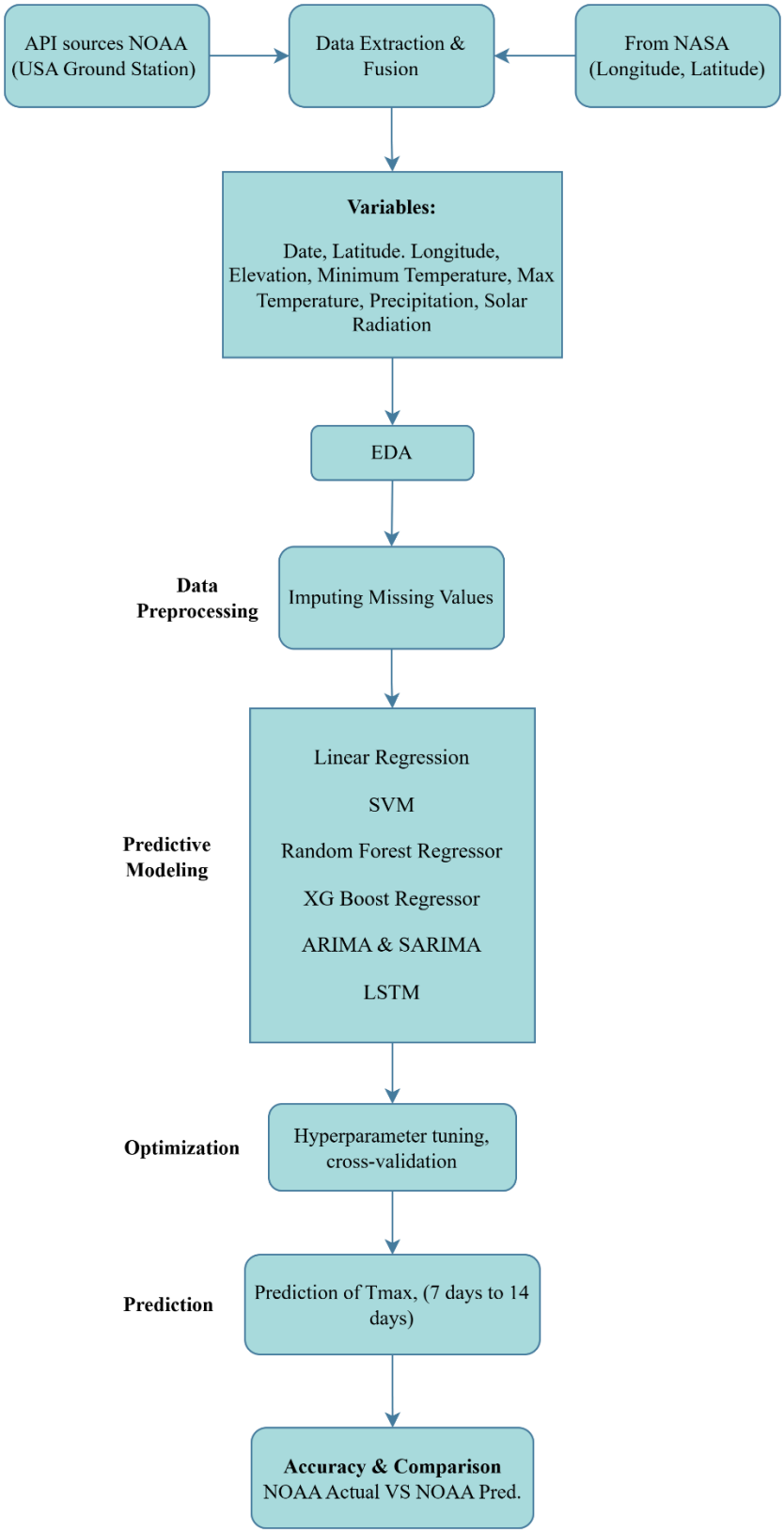
Challenges

Data Availability & Quality

- Access to high-resolution historical weather data from ground stations and satellites is limited or inconsistent.
- The local weather station has incomplete records, with missing data for several key variables over the past ten years.
- The dataset contains missing values and inconsistencies that require careful preprocessing and validation.
- Potential biases in collected data due to sensor errors, calibration issues, or environmental interference.

Methodology Overview

The core goal of this project is to accurately forecast temperature (TMAX) 7 days to 14 days in advance using local ground station data. To achieve this, a machine learning pipeline was designed to support temporal prediction with spatial awareness. The methodology integrates traditional data science techniques with domain-specific weather forecasting logic.


API sources NOAA (USA Ground Station)
Data Extraction & Fusion
From NASA (Longitude, Latitude)
Variables:
Date, Latitude. Longitude, Elevation, Minimum Temperature, Max Temperature, Precipitation, Solar Radiation
EDA
Data Preprocessing
Imputing Missing Values
Predictive Modeling
Linear Regression
SVM
Random Forest Regressor
XG Boost Regressor
ARIMA & SARIMA
LSTM
Optimization
Hyperparameter tuning, cross-validation
Prediction
Prediction of Tmax, (7 days to 14 days)
Accuracy & Comparison
NOAA Actual VS NOAA Pred.

Dataset Overview

This project initially started with the NOAA dataset provided which contains approximately 176,054 rows of daily meteorological data from **8 weather stations in Texas(near Dallas)** spans from **January 1, 2015, to February 2, 2025**. We have added Column "COUNTY" to it.

Sources

1) NOAA (Provided by GaiaDhi for 8 counties in Texas)
https://www.noaa.gov/

2) NASA Power API :
https://power.larc.nasa.gov/api/temporal/daily/point?parameters=PRECTOTCORR&community=ag&longitude={longitude}&latitude={latitude}&start={start}&end={end}&format=JSON

Data Fusion

The dataset from NOAA is sufficient for temperature forecasting. While Solar Radiation & Precipitation data could enhance accuracy, though it is not essential for a 7/14-day weather prediction model. Incorporating these variables can significantly enhance the model's predictive accuracy as these additional variables help capture underlying seasonality, trends, and correlations that improve the model's ability to generalize. To incorporate this, we sourced solar radiation data from NASA for the corresponding locations and fused it with the NOAA dataset, resulting in a consolidated dataset for model development. This approach supports accurate, spatially aware temporal predictions.

Note: The *Solar Radiation* column was manually added after performing imputation to reduce work around time. Since index matching across different weather stations did not yield accurate results using Python, we utilized Excel functions to ensure correct alignment.

The Final dataset includes the following **variables**:

- **STATION** – Unique identifier for each weather station.
- **NAME** – Location of the station.
- **LATITUDE, LONGITUDE, ELEVATION** – Geographical attributes of the station.
- **DATE** – Date of recorded weather observations.
- **PRCP (Precipitation)** – Daily recorded precipitation (in inches).
- **TMAX (Maximum Temperature)** – Highest temperature recorded for the day (in °F).
- **TMIN (Minimum Temperature)** – Lowest temperature recorded for the day (in °F).
- **RADN (Solar Radiation)- S**olar energy received at the station (W/m²) or (MJ/m²).

Exploratory Data Analysis (EDA)

A detailed EDA was performed using Python libraries like pandas, matplotlib, and seaborn to examine trends in TMAX, TMIN, PRCP, and spatial-temporal factors. Significant missing data, especially in temperature fields for counties. The **DATE** field was converted from object type to datetime for temporal analysis and feature extraction.

COUNTY	Number_of_Stations	Number_of_Rows	Number of Missing Rows	Missing Percentage	Missing PRCP	Missing TMIN	Missing TMAX
Brazos County	32	34171	30488	89.221855	774	30488	30488
Burleson County	7	8317	5587	67.175664	319	5307	5307
Grimes County	8	7134	7134	100.000000	82	7134	7134
Leon County	6	5227	1601	30.629424	191	1598	1561
Madison County	3	3919	598	15.258995	125	476	471
Milam County	37	48129	45563	94.668495	588	45491	45492
Robertson County	5	10176	6759	66.420991	104	6688	6688
Washington County	34	58981	50223	85.151150	1355	49833	49820

We noticed that Missing rows volume is high for the counties Due to a high proportion of missing values, especially in temperature fields (~83%) we decided to filter the dataset to focus on stations with more reliable data.

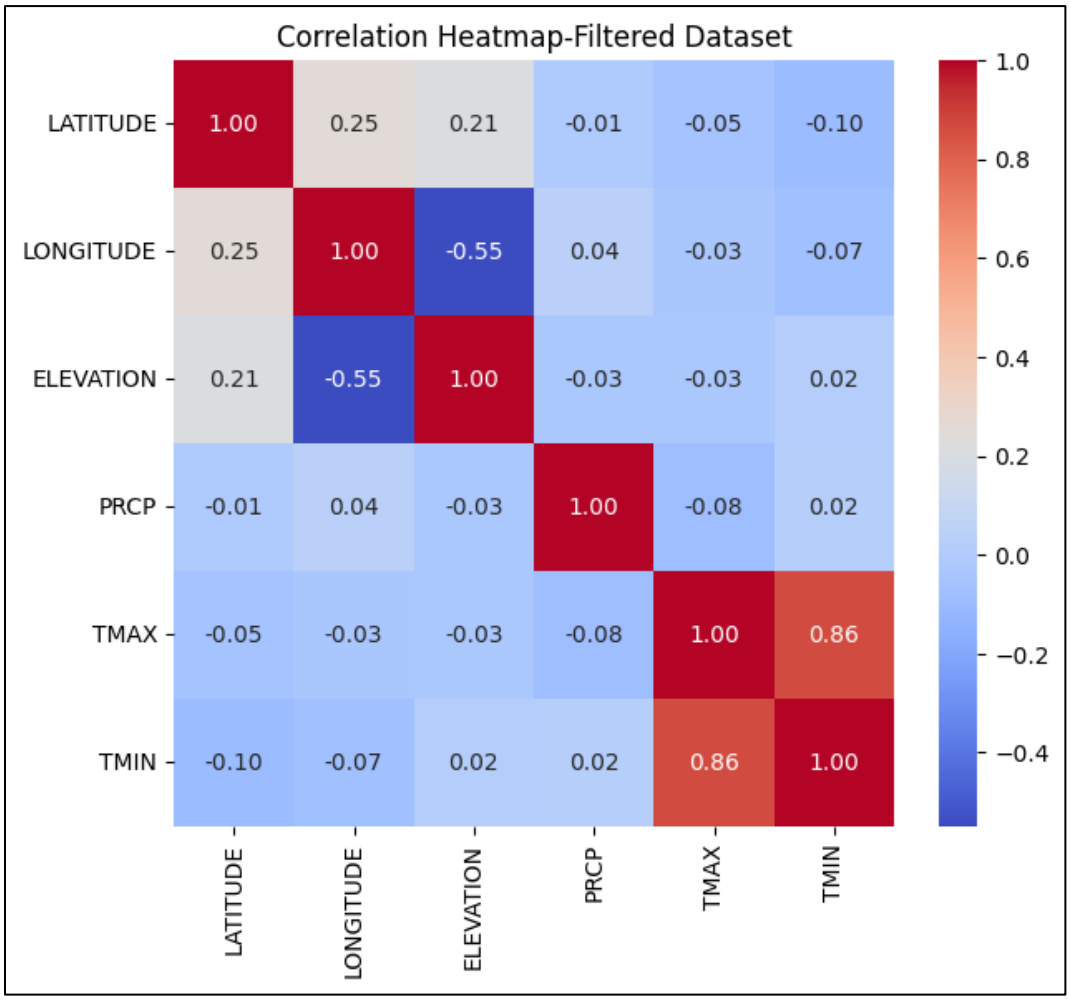
We filtered the data county-wise by **weather stations** to analyze missing value behavior and data availability. From this exploration, we observed:

	COUNTY	STATION	Total Rows	Missing Rows (Any)	Missing % (Any)	Missing PRCP	Missing TMIN	Missing TMAX
0	Brazos County	USW00003904	3686	3	0.081389	1	3	3
1	Burleson County	USC00418446	3021	291	9.632572	285	11	11
2	Leon County	USC00411596	3672	46	1.252723	0	43	6
3	Madison County	USC00415477	3456	135	3.906250	122	13	8
4	Milam County	USW00000351	2686	120	4.467610	71	48	49
5	Robertson County	USW00053973	3637	220	6.048941	73	149	149
6	Washington County	USC00411048	3648	205	5.619518	168	146	152
7	Washington County	USC00419491	2204	142	6.442831	45	131	112
8	Washington County	USW00053928	3598	345	9.588660	344	25	25

Brazos County (Station-USW00003904) had almost complete data with very minimal missing values (0.08% missing).The Precipitation (PRCP) values were most frequently missing across stations. The Washington County had multiple stations with varying data quality, highlighting inconsistencies within the same county. This station-level evaluation helped us identify the most reliable data sources for modeling and determine imputation techniques.

Correlation Analysis

A correlation matrix was generated to evaluate relationships between numerical features, in that



TMAX ↔ TMIN: 0.86, Shows strong positive correlation, indicating shared seasonal trends.

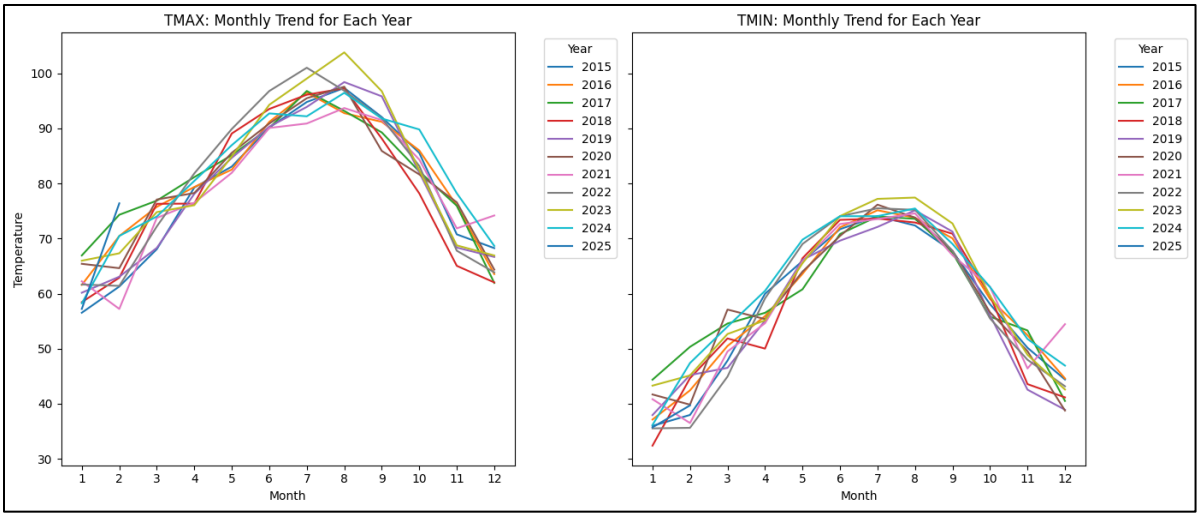
TMAX ↔ PRCP: -0.08, Shows weak negative correlation; higher precipitation slightly lowers Tmax.

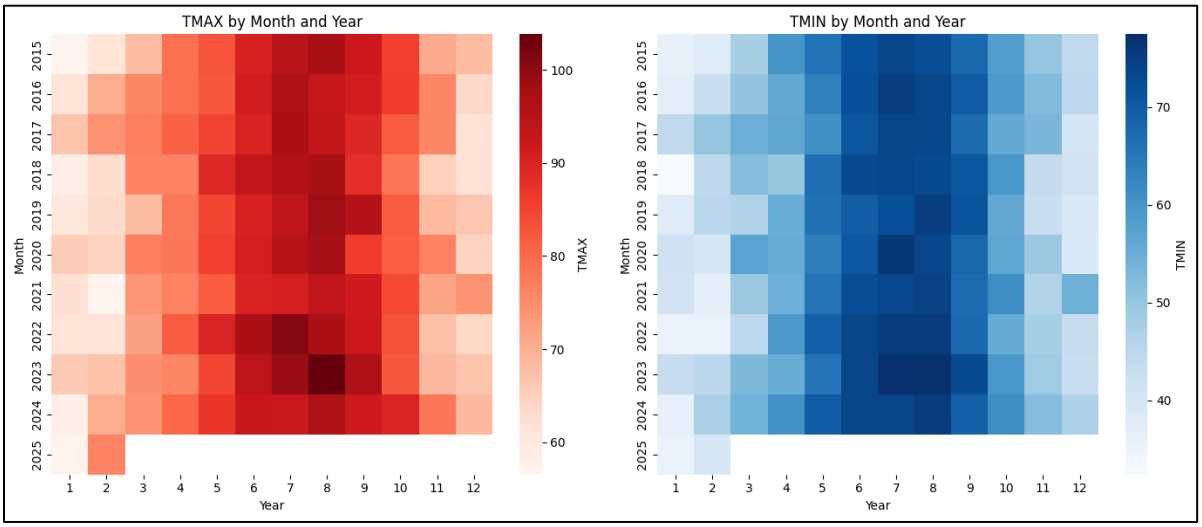
TMIN ↔ PRCP: 0.02, Shows Negligible correlation.

These results confirm that TMAX and TMIN move together, while precipitation and elevation have minimal influence on temperature in the filtered dataset.

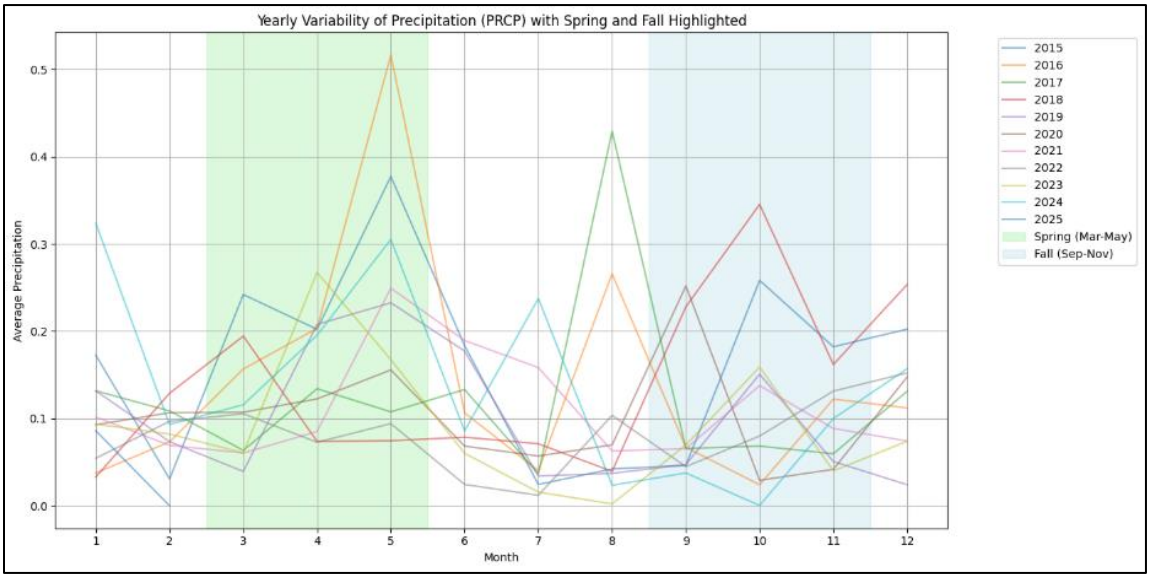
Pattern Analysis

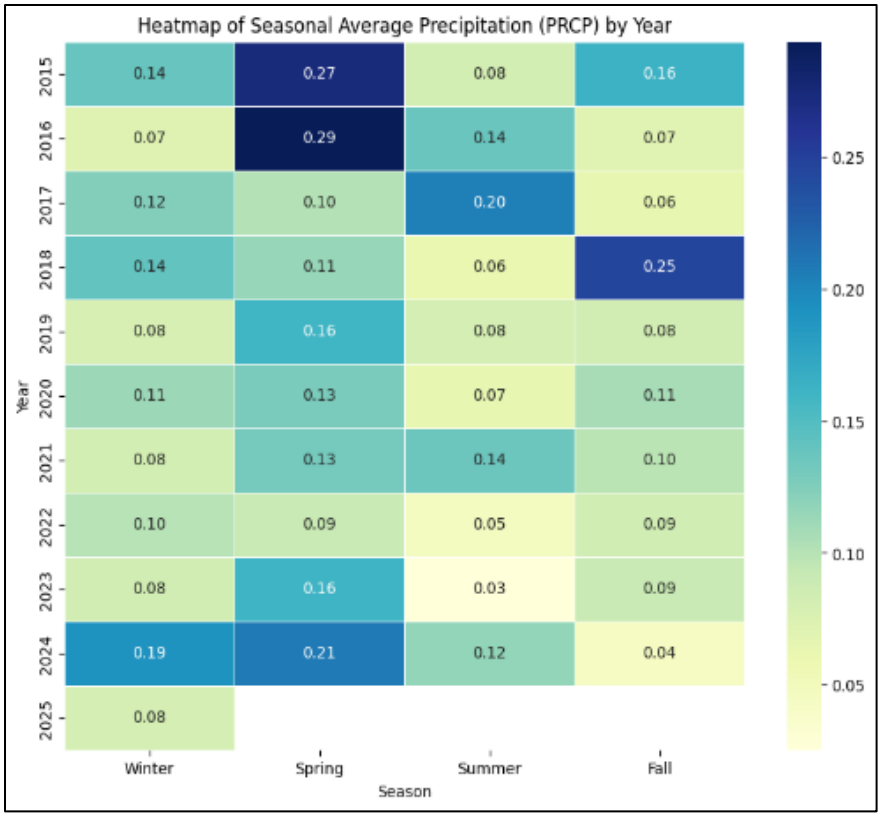
Analysis of the filtered dataset (~29,600 records) revealed minimal missing values in TMAX (~515), TMIN (~569), and PRCP (~1,109). Trends were assessed and clear seasonal patterns were observed Temperature peaked during summer (June–August) and dropped in winter (Dec–Feb).



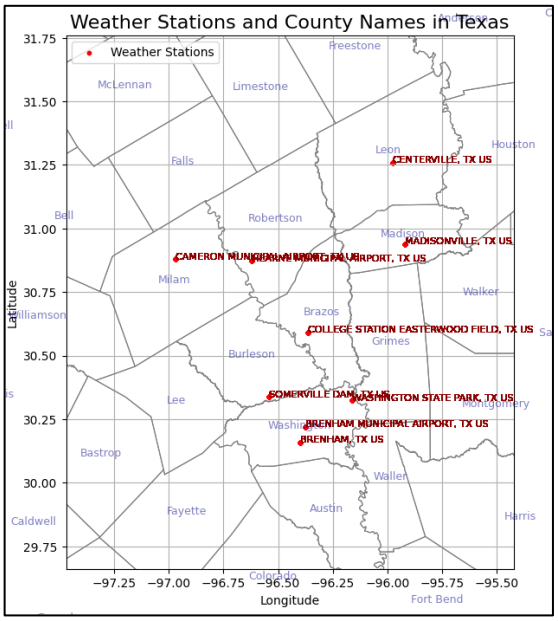


Precipitation showed yearly variability, with clusters in spring and fall.





Then we printed each station on a USA County map(Texas State) to see their locations for further analysis and processing.



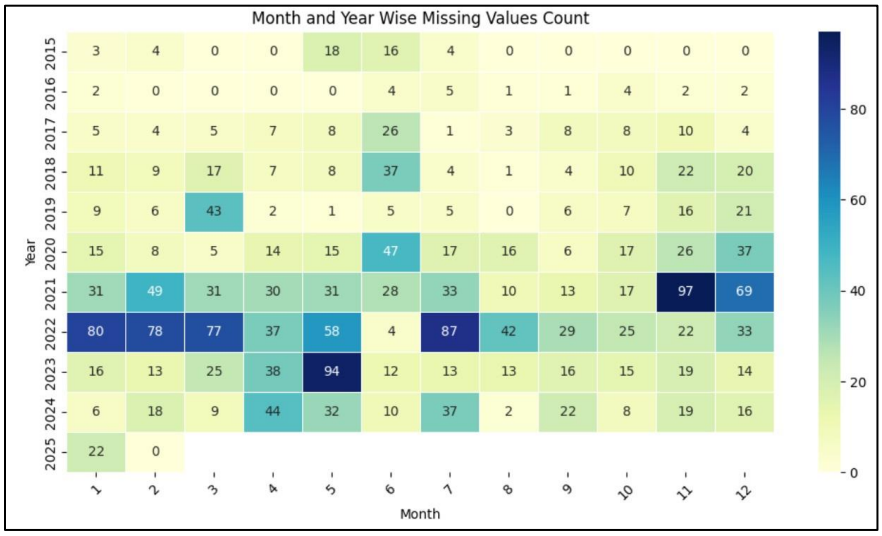
STATION	NAME
USW00003904	COLLEGE STATION EASTERWOOD FIELD, TX US
USC00418446	SOMERVILLE DAM, TX US
USC00411596	CENTERVILLE, TX US
USC00415477	MADISONVILLE, TX US
USW00000351	CAMERON MUNICIPAL AIRPORT, TX US
USW00053973	HEARNE MUNICIPAL AIRPORT, TX US
USW00053928	BRENHAM MUNICIPAL AIRPORT, TX US
USC00411048	BRENHAM, TX US
USC00419491	WASHINGTON STATE PARK, TX US

Data Preprocessing

After completing the assessment, we saved the cleaned and filtered station data into a file named **filtered_stations_data.csv** for further processing.

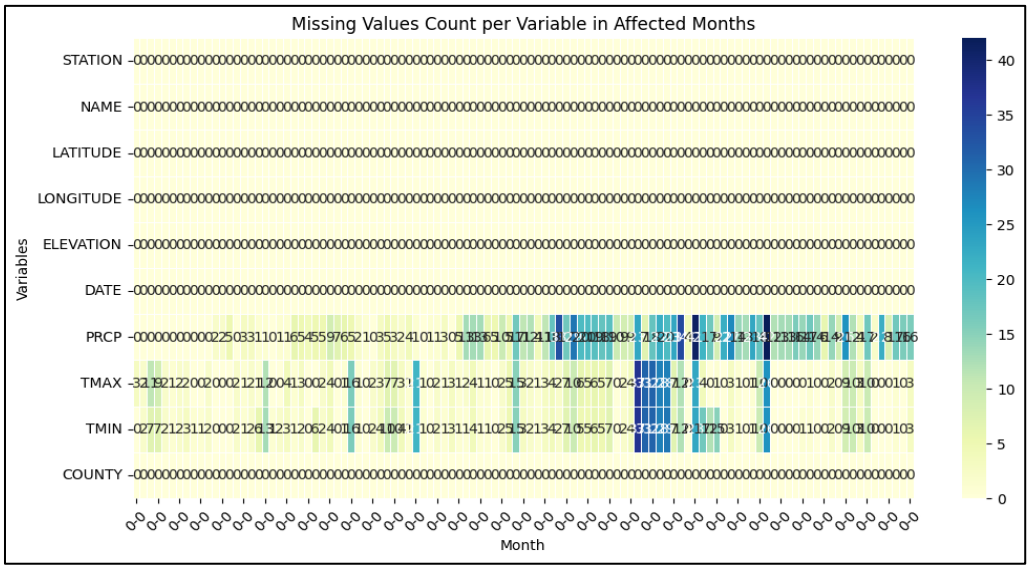
Handling Missing Values

We analyzed missing values **month-wise and year-wise** using a heatmap. The missing data was **comparatively low before 2020** but **increased significantly** after 2020, especially in 2021 and 2022. The annual missing value chart clearly shows a peak in 2022 with 572 missing records. This trend indicated that recent years required heavier imputation compared to earlier years.



	Year	Missing Values Count
0	2015	45
1	2016	21
2	2017	89
3	2018	150
4	2019	121
5	2020	223
6	2021	439
7	2022	572
8	2023	288
9	2024	223
10	2025	22

Then, we examined the missing data for each variable individually to better understand the missingness pattern. This step helped us decide on appropriate handling techniques, such as imputation or removal, depending on the variable's importance and the percentage of missing values.



Imputation

Handling missing values in TMAX and TMIN was crucial for preserving temporal and meteorological integrity. We did not remove the Anomalies as those are the crucial points for accurate forecasting.

Initial Approach – Seasonal Decomposition (STL)
STL was first used to decompose temperature time series into trend, seasonal, and residual components. While it maintained natural weather rhythms, it underperformed during extended missing periods or irregular patterns.

Final Approach – KNN Imputation (k=3)
KNN was selected for its superior accuracy in cross-validation. Using PRCP, LATITUDE, LONGITUDE, ELEVATION, and rolling/lags of TMAX and TMIN, it effectively captured spatial-temporal context and outperformed STL in RMSE. KNN provided a scalable and robust solution to impute missing data without distorting seasonal trends.

After imputing the data, it was exported to the csv file named **"imputed_stations_data.csv".** As mentioned earlier, The *Solar Radiation* column(From NASA) was manually added, and the file was renamed as **"imputed_all_stations_data.csv".**

Model Building

We developed Machine Learning models to predict TMAX over a short-term(7-14 days) forecast horizon. The Machine Learning Models we implemented are Linear Regression, SVM, Random Forest, XG Boost, ARIMA, SARIMA, LSTM. Finally, we compared the actual NOAA values with the predicted values and chose the best model for deployment.

We evaluated our model's performance using Mean Squared Error (MSE), R-Squared (R^2), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) to comprehensively assess its accuracy, goodness-of-fit, and prediction error magnitudes.

Approach

To ensure modeling consistency, we have selected **Brenham station (USC00411048)** as a pilot site due to its data completeness and we have 3 stations data from the Washington county. The objective is to identify the best-performing model that yields the highest prediction accuracy based on this station's data. Once the optimal model is determined, it will be applied to forecast conditions for other stations across the region.

Splitting Data into Training Set: 2015-2023 (9 years) & Testing Set: 2024 (1 year) for all models except for ARIMA ,SARIMA & LSTM we split Train: Test: Val as 80:10:10

Hyperparameter: GridSearchCV/ RandomizedSearchCV were the hyperparameters used for the models to identify the best model by having different sets of parameters

Linear Regression

Linear regression is a description of the relationship between an independent variable and one or several independent variables in a linear equation. It can be simple or multiple based on explanatory variables.

Linear Regression was not performing well, so used Lasso and Ridge Parameters as well which didn't perform well. Tried Linear Regression with hyperparameter tuning using RandomizedSearchCV and GridSearchCV.

Hyperparameter Tuning

Linear Regression with hyperparameter tuning using RandomizedSearchCV with parameters like

- **Tuned parameters:** fit_intercept, positive, copy_X
- **Cross-validation:** 3-fold
- **Iterations:** 10 with n_jobs=-1 for parallel tuning

Temporal Features

We extracted components from the DATE column to create new time-based features such as Month, DayOfYear, WeekOfYear, DayOfWeek, and a boolean IsWeekend. These features help the model capture:

- Seasonal patterns (e.g., summer vs. winter)
- Weekly cycles (e.g., weekday vs. weekend behavior)
- Long-term or cyclical trends throughout the year

Lag Features

Lag variables were generated for the previous 1 to 7 days for maximum temperature (TMAX), minimum temperature (TMIN), and precipitation (PRCP). These lagged features allow the model to learn from recent values and model short-term dependencies effectively.

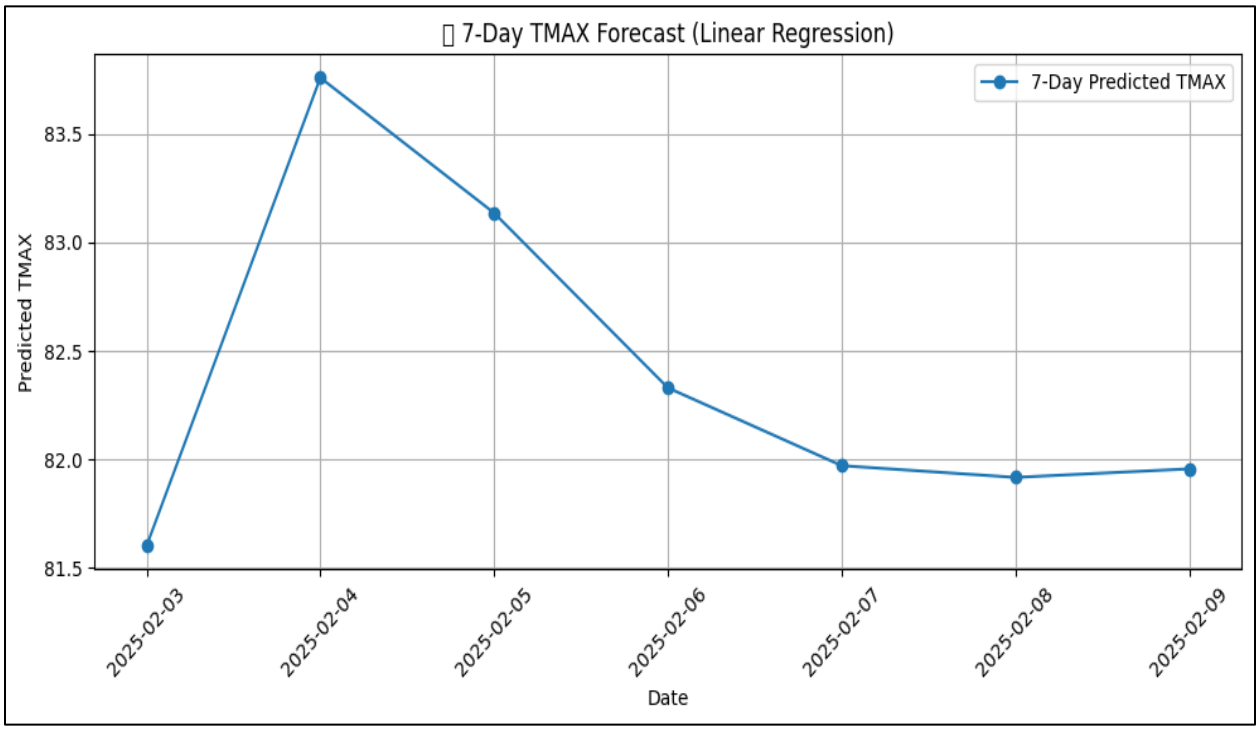
Rolling and Trend Features

- TMAX_3day_avg: captures the average of the past 3 days' maximum temperatures, smoothing short-term volatility.
- PRCP_3day_sum gives a short-term rainfall accumulation, which is important for modeling weather influence.
- TMAX_trend indicates temperature momentum (whether it's rising or falling) by computing the difference between the current and previous day's temperature.

After generating lagged and rolling features, rows containing NaN values (which result from the shifting and rolling operations) were removed to ensure a clean training set. These features significantly enrich the dataset, helping the model understand both seasonal patterns and current trends, which are essential for accurate and robust time series predictions.

```
✅ Train Set Performance:
R² Score: 1.0000, MAE: 0.0000, MSE: 0.0000, RMSE: 0.0000
```

```
📊 Test Set Performance (Full Test Forecast):
R² Score: 1.0000, MAE: 0.0000, MSE: 0.0000, RMSE: 0.0000
```



	DATE	Predicted_TMAX
0	2025-02-03	81.600000
1	2025-02-04	83.760000
2	2025-02-05	83.136000
3	2025-02-06	82.329600
4	2025-02-07	81.970560
5	2025-02-08	81.916416
6	2025-02-09	81.955738

Perfect performance(R^2=1) on both the train and test sets indicates overfitting or data leakage, caused by test features unintentionally containing future information from the target variable. Such leakage leads to unrealistically high accuracy, masking the model's true predictive ability.

Support Vector Regression (SVR)

Support Vector Machine Regression (SVR) is Support Vector Machine for making continuous value predictions. It seeks to find a function that deviates by few margins (epsilon) from actual values and is as flat as possible. SVR utilizes kernel functions to operate with linear as well as non-linear relationships.

Lagged and Trend Features

We created lag features for 1 to 7 days for TMAX, TMIN, and PRCP to capture recent historical patterns. Additionally, we engineered TMAX_3day_avg (3-day rolling average), PRCP_3day_sum (3-day rolling sum), and TMAX_trend (day-over-day difference). These features enhance the model's ability to recognize short-term behavior and current trends in the data.

Feature Selection

The final model used a diverse set of features including TMIN, PRCP, SOLAR RADIATION, temporal features, rolling averages, trends, and 1–7 day lagged variables. This multi-dimensional input helps the model capture both short-term fluctuations and long-term trends in the data.

Hyperparameter Tuning

We used Support Vector Regression (SVR) with the RBF kernel and tuned hyperparameters (C and gamma) using RandomizedSearchCV like

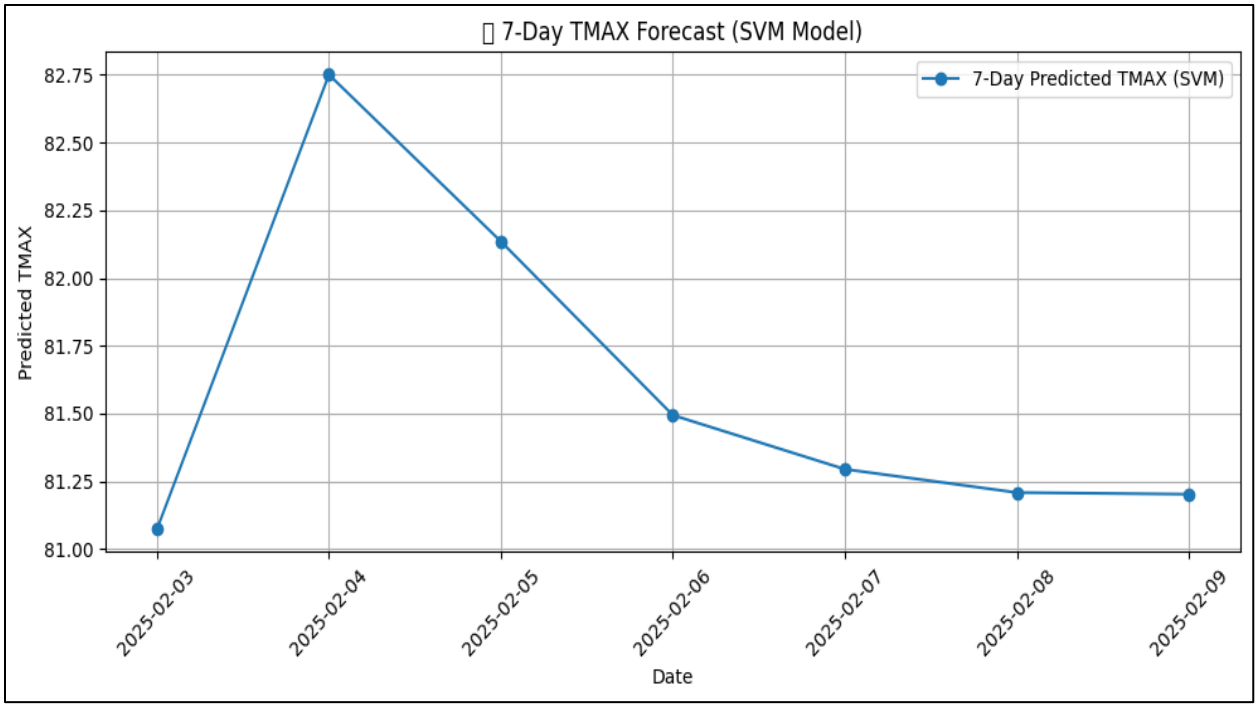
- **Tuned parameters:** C: [1, 10],gamma: ['scale', 0.1],kernel: ['rbf']
- **Cross-validation:** 3-fold
- **Iterations:** 4, random_state=42, verbose=1, n_jobs=-1

After training, the best model was selected based on the minimum mean squared error from the search results. The best-performing SVR model was applied to the test set

```
✅ Train Set Performance:
R² Score: 0.9975, MAE: 0.3051, MSE: 0.4676, RMSE: 0.6838
```

```
📊 Test Set Performance (Full Test Forecast):
R² Score: -0.2704, MAE: 10.1781, MSE: 217.9340, RMSE: 14.7626
```

	DATE	Predicted_TMAX
0	2025-02-03	81.073374
1	2025-02-04	82.750513
2	2025-02-05	82.135019
3	2025-02-06	81.494052
4	2025-02-07	81.294725
5	2025-02-08	81.208608
6	2025-02-09	81.202506



The Support Vector Regression (SVR) model demonstrated powerful performance on the training data, achieving a high R^2 value; however, it performed poorly on the test set, as evidenced by a negative R^2 score. This indicates that the model's predictions on unseen data were worse than simply predicting the mean, suggesting significant overfitting. The 7-day forecast captured the expected trend with a peak around February 4–5, followed by a gradual tapering, but the generalization ability of the SVM model was weak. While the forecasted values appeared in shape, they lacked robustness when evaluated on new data. The poor test performance is due to overfitting caused by limited or non-representative training data, along with the SVR model's sensitivity to noise and the potential need for better feature scaling.

ARIMA (Auto Regressive Integrated Moving Average)

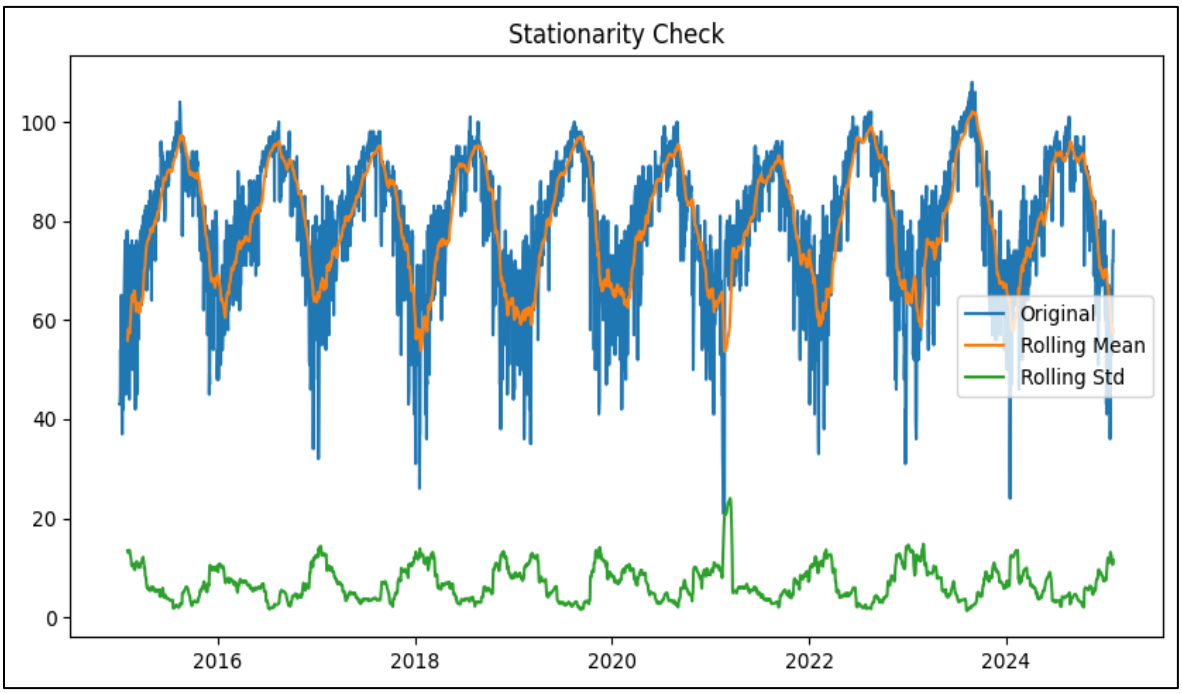
ARIMA is a time series forecasting technique that exhibits trends and autocorrelations in non-seasonal data. It combines autoregressive (AR), differencing (I), and moving average (MA) components to capture temporal patterns. ARIMA is widely used in weather forecasting for its effectiveness in modeling time-dependent data.

ARIMA difference a series to make it stationary and subsequently fits it using autoregressive (AR) and moving average (MA) components. The "integrated" (I) part picks up this different operation. Selection of appropriate differencing order (d), AR (p), and MA (q) is through visual, ACF/PACF plot inspection, and statistical tests like ADF for employing unit-root tests and the model is selected with small information criteria such as AIC. To achieve maximum performance and stability, we

used an 80/10/10 train-test-validation split that worked better than the 90/10 by generating more stable results.

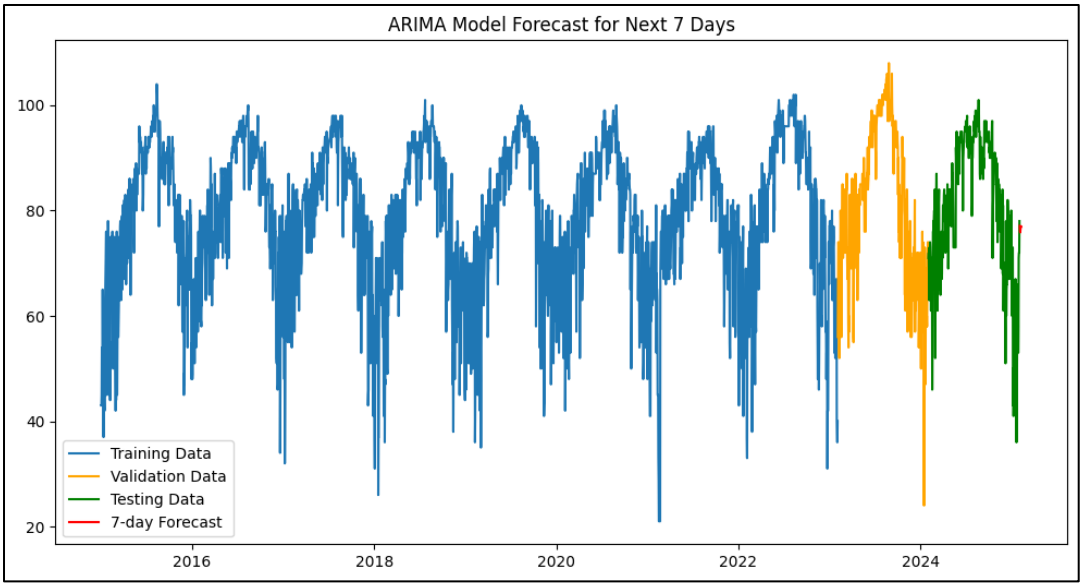
```
Best Model: Order (4, 0, 0) with Test MSE: 167.333, Test MAE: 10.714, Test RMSE: 12.936, Test R²: -0.001,
```

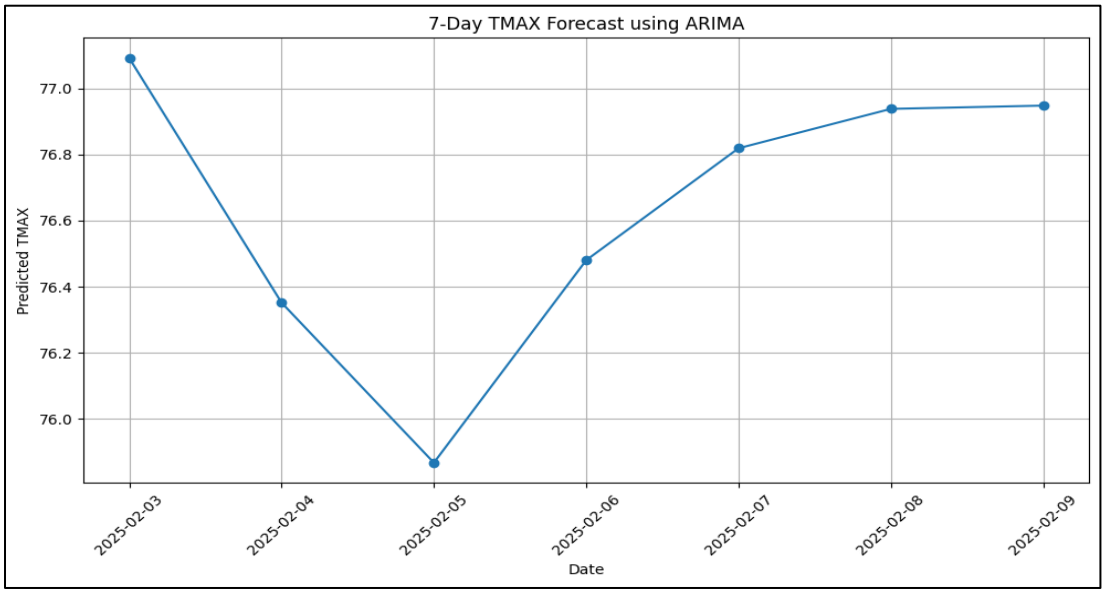
Despite its limitations, the ARIMA model provides a structured starting point for time series forecasting. With MSE (167.33), RMSE (12.94), and MAE (10.71) for the (4,0,0) model, we establish clear benchmarks for improvement, while the near-zero R^2 underscores substantial room to enhance predictive performance. A stationarity checks confirmed differencing orders of 0 and 1. This diagnostic feedback directs future adjustments to p, d, and q values, beginning with (1,1,1), ensuring that refinements remain data-driven rather than arbitrary. Overall, the current results offer a measurable foundation for iterative model optimization.



Hyperparameter Tuning

For the hyperparameter tuning, different ARIMA model specifications were attempted, representing different combinations of (p, d, q) orders. The candidate orders are parameters such as (1, 1, 0), (1, 1, 1), (2, 1, 2),(0, 1, 1),(2, 1, 0),(1, 0, 1),(1, 0, 0),(1, 1, 4),(1, 1, 6),(2,1,4),(2, 1, 6),(3, 1, 3) up to (4, 0, 0). These parameters were chosen to experiment with different autoregression orders (p), differencing orders (d), and moving average orders (q) to detect different types of patterns in the time series to choose (4,0,0) as the best model by keeping AIC as the main criterion after confirming with R Studio for model performance.





DATE	Predicted_TMAX
2025-02-03	77.092062
2025-02-04	76.351638
2025-02-05	75.867126
2025-02-06	76.480237
2025-02-07	76.819126
2025-02-08	76.938306
2025-02-09	76.948190

The 7-day predictions initially show a downward trend, followed by an upward shift, which differs from the trend observed in the actual NOAA dataset. In the real data, the trend increases initially and then decreases toward the end.

SARIMA (Seasonal Auto Regressive Integrated Moving Average)

SARIMA is a time series prediction algorithm, which incorporates seasonality into ARIMA. SARIMA uses autoregressive (AR), differencing (I), and moving average (MA) with their seasonal variants. SARIMA takes the form of SARIMA(p, d, q)(P, D, Q, s), where the seasonality period is represented as 's'. SARIMA can model both short-term and seasonality patterns within data. SARIMA is universally used in the field of weather prediction.

Limitations

When we tested the SARIMA model with R, the best configuration identified was (4, 0, 0), (0, 1, 0, 365), which provided the best results. However, when attempting the same model in Python, it became too memory-intensive and crashed due to system limitations. To work within these constraints, we decided to evaluate with a seasonal period of 26 days instead of 365 days, representing a biweekly cycle. This adjustment allowed us to select the best model without exceeding hardware limitations while still capturing significant seasonal patterns, including both upward and downward trends. We used an 80/10/10 train-test-validation split that worked better than the 90/10 by generating more stable results

```
Best Model: Order (0, 1, 1), Seasonal Order (0, 1, 1, 26) with Test MSE: 239.629, Test MAE: 13.233, Test RMSE: 15.480, Test R²: -0.434
```

SARIMA model yielded an MSE of 239.63, RMSE of 15.48, and MAE of 13.23, indicating moderate prediction errors. Although the R^2 value is negative (-0.434), suggesting limited explanatory power, the model still captures seasonal patterns well and offers a useful foundation for future tuning and improvement.

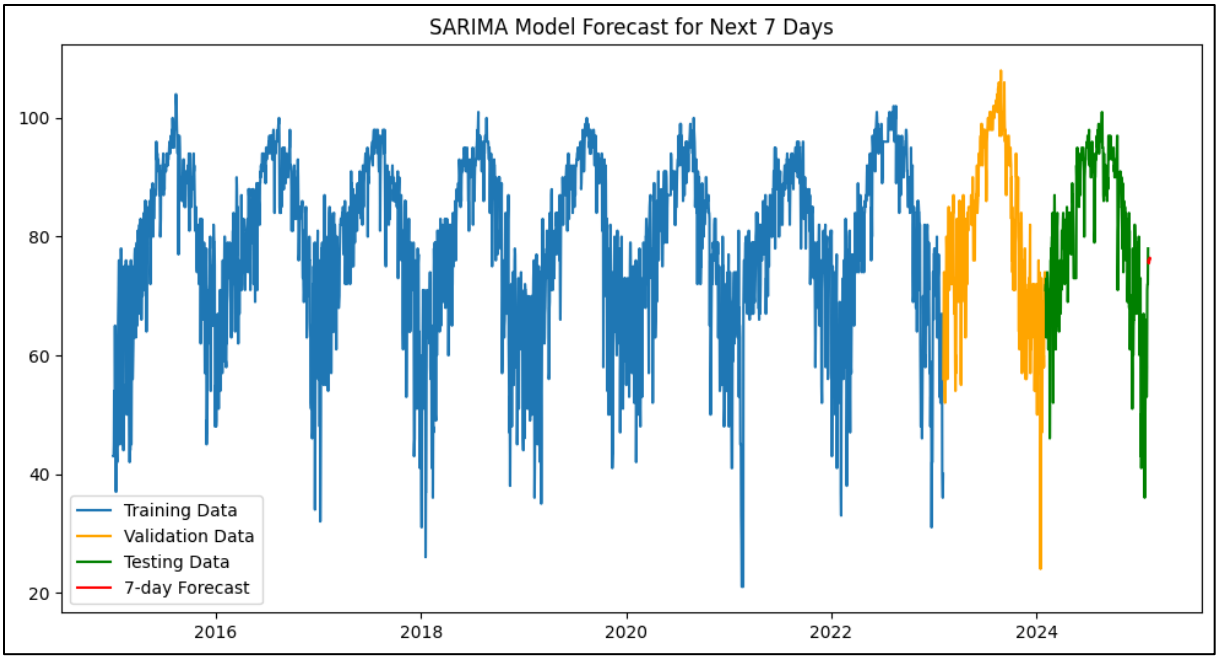
Hyperparameter Tuning

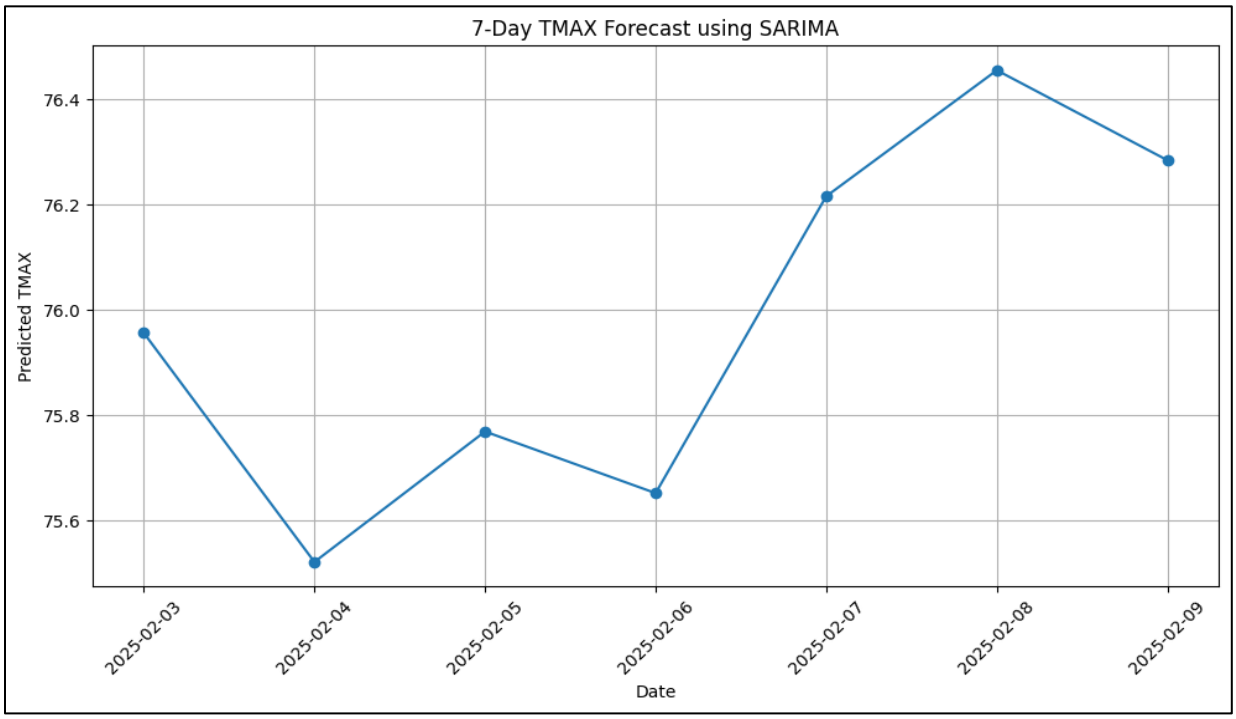
We evaluated the range of SARIMA model configurations with different combinations of non-seasonal and seasonal parameters. The candidate orders included:

- ((0, 1, 1), (0, 1, 1, 26))
- ((1, 1, 1), (1, 1, 1, 26))
- ((2, 1, 2), (1, 1, 1, 26))
- ((1, 1, 1), (0, 0, 0, 26))
- ((4, 0, 0), (0, 1, 0, 26))

After conducting the tests in RStudio we selected the configuration (0, 1, 1), (0, 1, 1, 26) with AIC as the best criterion for model forecasting, as it provided the most reliable results within the system's limitations.

DATE	Predicted_TMAX
2025-02-03	75.956277
2025-02-04	75.521234
2025-02-05	75.769049
2025-02-06	75.652016
2025-02-07	76.215920
2025-02-08	76.455266
2025-02-09	76.284365





The SARIMA model forecasted the 7-day TMAX values, displaying a general downward trend until February 6, 2025, followed by a consistent upward rise through February 9, 2025. Although the model captures seasonal movement patterns well, it fails to align precisely with the actual NOAA data, indicating the need for further tuning or alternative modeling approaches to better match real-world variations.

LSTM (Long Short-Term Memory)

LSTM is a recurrent neural network (RNN) that is used to learn long-term dependencies from sequence data. LSTM prevents vanishing gradients that occur in the case of normal RNNs through the assistance of memory cells and gates (input, forget, and output gates) to manage information flow. LSTMs are best suited for prediction of time series. They can process sequences of varying lengths and remember important patterns for extended periods. LSTMs are well-suited to process information with temporal relationships.

The following three lag strategies were implemented:

(a) 5-Day Lag Model

- Features: TMAX values from the past 5 days.
- This model captures short-term weather dynamics and is suitable for near-future forecasting due to its simplicity and responsiveness to recent patterns.

(b) 30-Day Lag Model

- Features: TMAX values from the past 30 days.
- Designed to leverage longer-term patterns and capture broader seasonal and climatic variability, although it introduces a higher-dimensional feature space.

(c) 5-Day + 3-Year Lag Model

- Features: TMAX from past 5 days + the same calendar days from 3 years prior (days 1095, 1096, and 1097 back).
- This hybrid model aims to incorporate both recent and long-term seasonal information to account for cyclical climate trends such as annual temperature repetition.

Each lag model was generated by shifting the `TMAX` time series to create lagged input features. The forecast horizon was set to 14 days.

Hyperparameter Tuning

LSTM-64 units, Activation functions like 'relu','linear',Kernel Initializer like 'he_normal','adam' optimizer, '0.001' as Learning Rate, Number of epochs as '100'

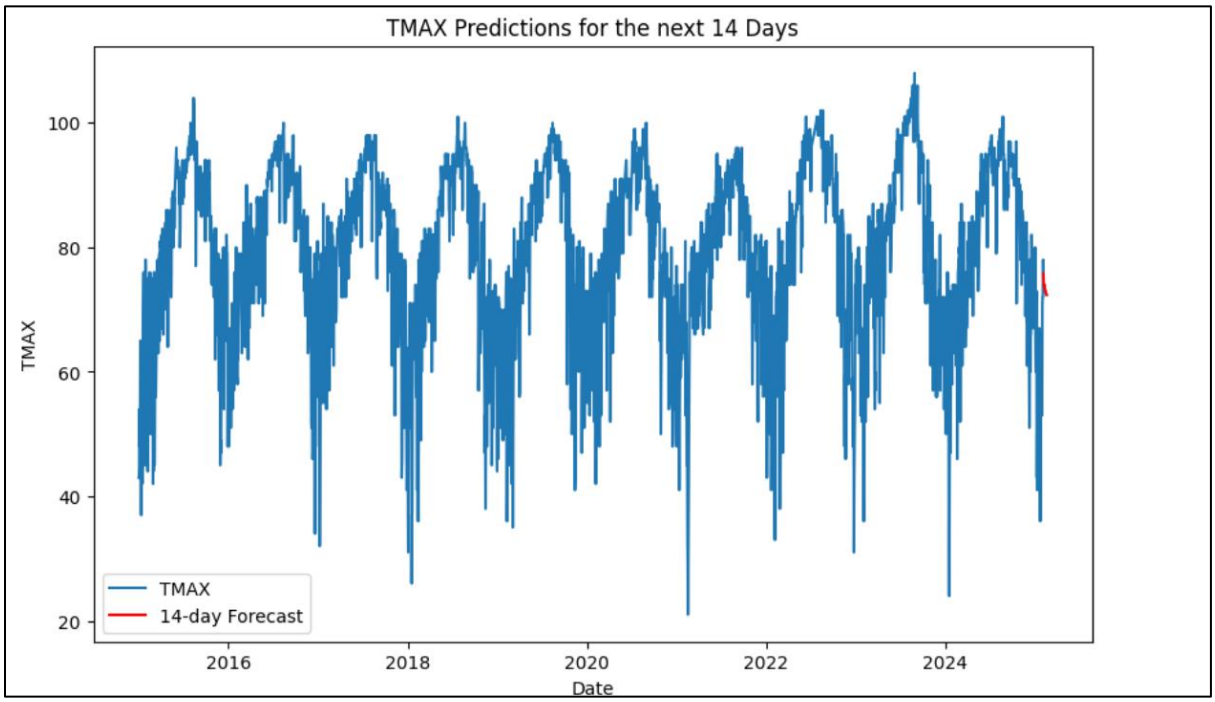
The performance of each lag-based LSTM model on the test set is summarized in the table below.

Model	MAE	RMSE
5-Day Lag	3.62	5.28
30-Day Lag	4.58	6.20
5-Day + 3-Year Lag	4.57	6.19

The results indicate that the 5-Day Lag Model significantly outperforms the other two models in terms of both MAE and RMSE. This suggests that short-term historical data is more effective at capturing the temporal dependencies necessary for accurate temperature forecasting over a 14-day horizon. The 30-Day Lag Model and 5-Day + 3-Year Lag Model, despite incorporating a larger

temporal context and seasonal history, did not yield improved accuracy. Instead, these models exhibited higher error rates, which may be attributed to the added noise, overfitting, or the reduced relevance of long-term data for short-to-mid-term forecasts. This finding reinforces the notion that more data is not always better, especially when it does not directly contribute to short-term predictability. The added complexity of the extended lag features may have introduced diminishing returns in model learning.

Then we took 5-Day Lag model again re-iterate with 1000 epochs to find the best fit model and we predicted TMAX for next 7 to 14 Days.



Date	Predicted_TMAX
2025-02-03	75.770683
2025-02-04	74.046776
2025-02-05	73.812378
2025-02-06	74.012962
2025-02-07	73.853561
2025-02-08	73.513252
2025-02-09	73.264359

Best Model Metrics
MAE : 4.6634
RMSE : 6.4413

While the final model achieved lower error metrics (MAE: 4.6634, RMSE: 6.4413) on the validation set, the forecasted TMAX values show an unrealistic continuous decrease across the 7-day period. This suggests that, despite the improved numerical performance, the model fails to fully capture the true seasonal fluctuations observed in the actual NOAA data, indicating potential underfitting or missing seasonality components.

Random Forest Regressor

Random forests are an ensemble learning method that builds multiple decision trees for classification and regression tasks. Here for regression, it is the average of all tree predictions. Random forests help reduce overfitting, a common issue with individual decision trees. Random Forest is particularly well-suited for structured weather data due to its ability to manage multicollinearity and automatically model complex interactions between features.

Lagged and Temporal Features

Lagged features for 1 to 7 days were generated for TMAX, TMIN, and PRCP to model short-term temporal dependencies and capture recent weather trends. In addition, temporal features such as Month, DayOfYear, WeekOfYear, DayOfWeek, and a binary IsWeekend indicator were engineered from the date to account for seasonal and weekly patterns.

Rolling Statistics and Trend Features

To enhance short-term forecasting, rolling statistics were added, including:

- TMAX_3day_avg: 3-day moving average of TMAX
- PRCP_3day_sum: 3-day cumulative precipitation
- TMAX_trend: Difference between today and yesterday TMAX, indicating short-term temperature momentum.

These features helped the model capture both immediate fluctuations and broader seasonal variations in temperature data.

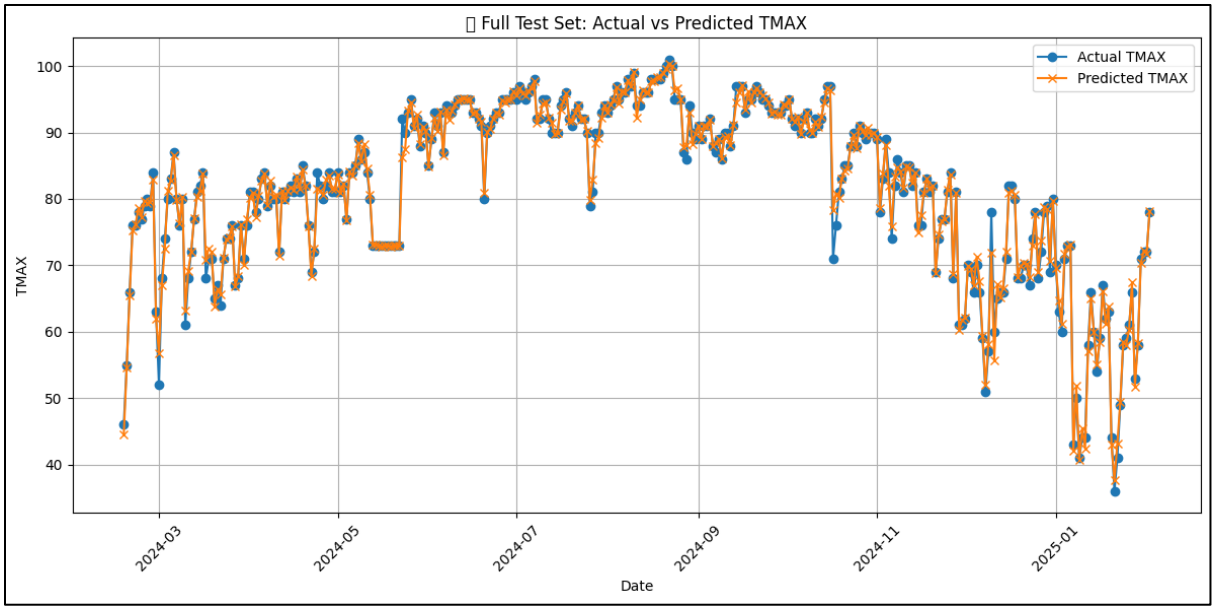
Hyperparameter Tuning

We used a Random Forest Regressor and tuned hyperparameters using RandomizedSearchCV to optimize the model's predictive performance. The following parameters and setup were used:

- Method-1: 'n_estimators': list(range(100, 400, 50)), 'max_depth': [5, 7, 9, 11], cv=3, n_jobs=-1, n_iter=20
- Method-2: 'n_estimators': list(range(100, 500, 50)), 'max_depth': [5, 10, 15], 'min_samples_split': [2, 5, 10], cv=3, n_jobs=-1, n_iter=10

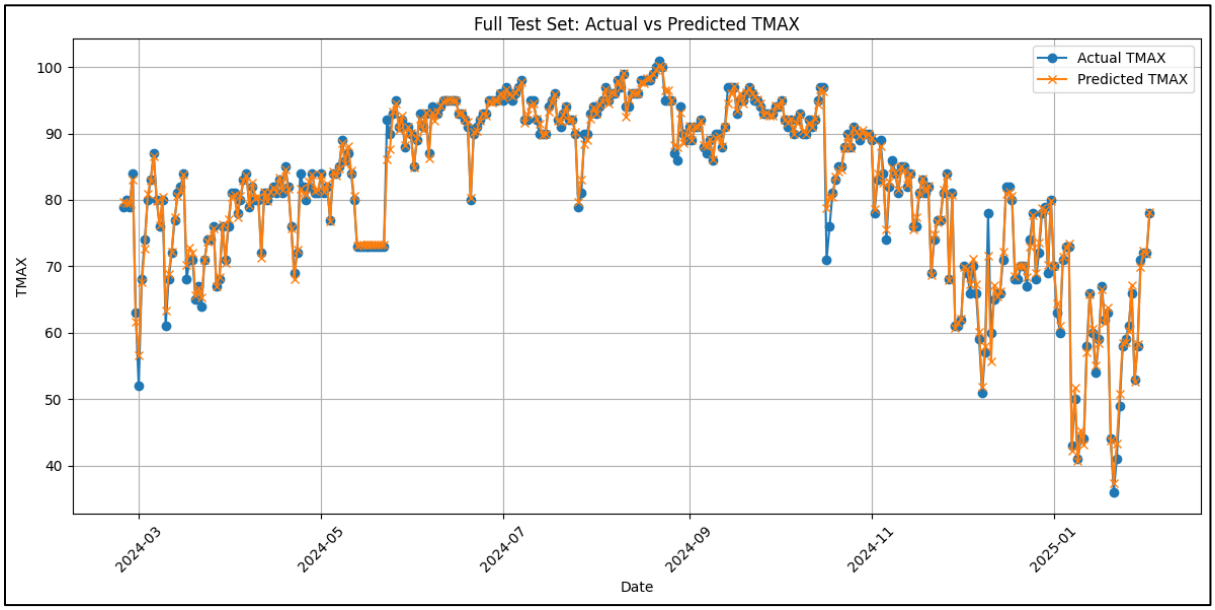
Data Split: 90% for training and 10% for validation, without shuffling to preserve the temporal order

The Random Forest model demonstrated strong generalization with minimal overfitting, achieving high R^2 and low error values on both the training and test datasets. The model closely follows the actual observations with minimal deviation, although minor discrepancies are noticeable during periods of rapid temperature changes. The 7-day to seasonal patterns were well tracked, reflecting the model's ability to generalize to unseen data. This robust performance indicates that the Random Forest model avoided overfitting, due to ensemble averaging and effective feature engineering.



Analysis Excluding Solar Radiation

Excluding solar radiation did not significantly impact the model's accuracy, demonstrating that a well-crafted set of lagged and temporal features can effectively capture the necessary patterns for TMAX forecasting. However, solar radiation may still enhance performance in more complex or variable climatic settings. The strong results also reaffirm the Random Forest model's suitability for time-series forecasting when temporal features are properly engineered.



Method-1: Forecasting of TMAX Using Extended Lag Features from manual inputs

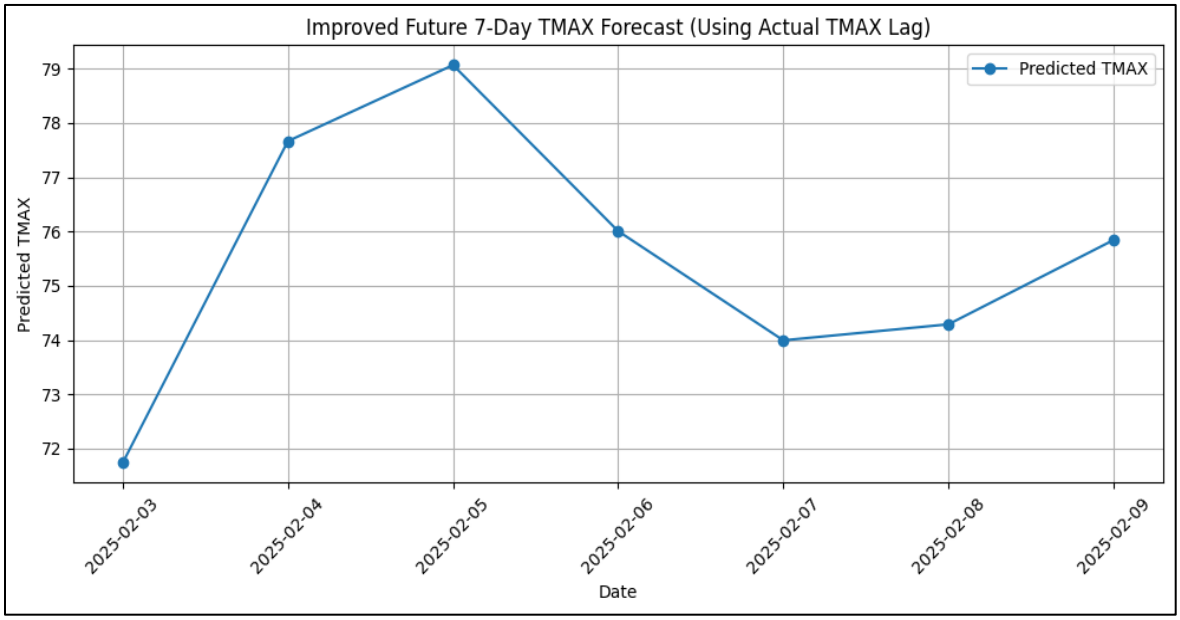
Lag features for the previous 14 days were created for TMAX, TMIN, and PRCP. Solar radiation and rolling statistics were excluded to evaluate model robustness under reduced feature complexity. A 7-day TMAX forecast was generated by manually inputting future TMIN and PRCP values. For each forecasted day, the model utilized the latest actual TMAX observations from the test set as lag features, avoiding recursive prediction and minimizing error propagation.

```
🔍 Train Set Performance:
R²: 0.975
MAE: 1.7049
MSE: 5.1165
RMSE: 2.262
```

```
💉 Test Set Performance:
R²: 0.8888
MAE: 3.0123
MSE: 18.9264
RMSE: 4.3504
```

📈 Improved Future 7-Day TMAX Forecast:

	DATE	TMIN	PRCP	Predicted_TMAX
0	2025-02-03	45	0.0	71.737432
1	2025-02-04	65	0.0	77.665268
2	2025-02-05	68	0.0	79.067810
3	2025-02-06	67	0.2	76.006769
4	2025-02-07	54	0.0	73.994330
5	2025-02-08	50	0.0	74.290023
6	2025-02-09	52	0.0	75.843665



The improved 7-day TMAX forecast, generated using actual lagged TMAX values, shows a more realistic trend with both upward and downward movements. Unlike earlier models, this forecast better captures the natural variability observed in real temperature data, indicating a significant enhancement in the model's ability to reflect seasonal fluctuations.

Method-2: Forecasting TMAX Using Recursive Multi-Stage Feature Engineering

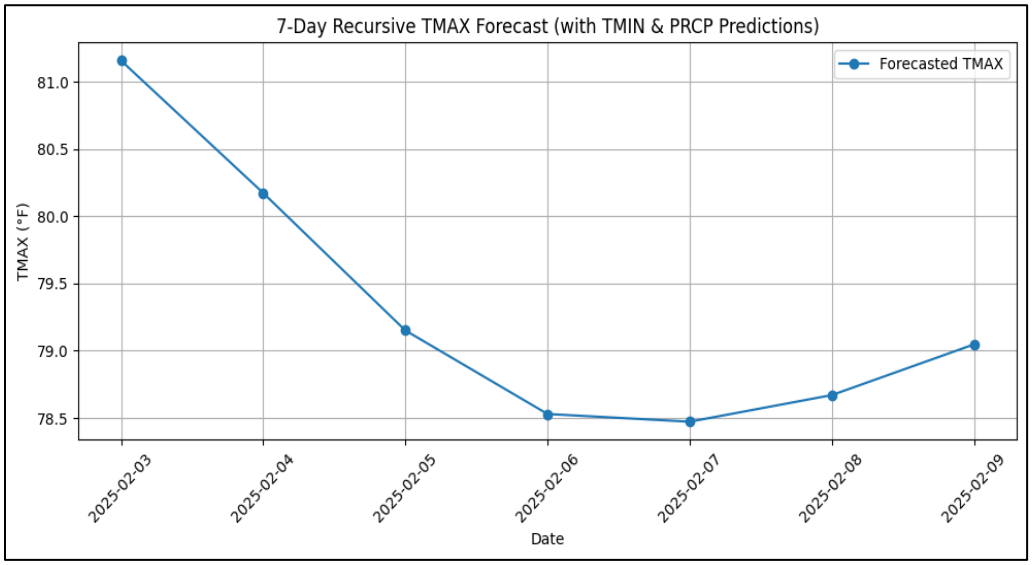
The 7-day forecasting process employed a recursive approach, where the most recent seven days of observed or predicted data were used as lagged inputs for future predictions. For each forecasted day, TMIN and PRCP values were predicted sequentially, and these predictions, combined with static solar radiation values, were then used to forecast TMAX. After each day's forecast was generated, the dataset was iteratively updated by incorporating the new predictions, ensuring that the inputs for the subsequent day's forecast reflected the latest available information.

```
🔍 TMAX Model – Train Set Performance:
R²: 0.9989
MAE: 0.252
MSE: 0.213
RMSE: 0.4616
```

```
🧪 TMAX Model – Test Set Performance:
R²: 0.9949
MAE: 0.5098
MSE: 0.8311
RMSE: 0.9117
```

The model performs exceptionally well on the training data and generalizes on test data, capturing temperature variations with low mean error.

	DATE	Predicted_TMAX
0	2025-02-03	81.157932
1	2025-02-04	80.173264
2	2025-02-05	79.151261
3	2025-02-06	78.529832
4	2025-02-07	78.472511
5	2025-02-08	78.670381
6	2025-02-09	79.047454



The Random Forest model predicted 7-day TMAX values with moderate accuracy. While it captures the general temperature range, the model consistently underestimates higher actual TMAX values and overestimates lower ones. The predicted temperatures show less variation

compared to the observed NOAA values, indicating that while Random Forest smooths out predictions, it struggles to fully capture the dynamic fluctuations in the actual data.

The use of recursive modeling allows for effective chaining of weather dependencies, especially the influence of temperature and precipitation on daily maximum temperature. However, recursive methods can introduce error propagation, particularly when earlier forecasts are inaccurate. In this implementation, the use of robust tree-based models and smoothed features (e.g., rolling averages) helps mitigate instability.

XG Boost Regressor

XGBoost Regressor is a gradient boosting-based machine learning algorithm that is fast and efficient. It builds an iterative sequence of decision trees, each of which attempts to minimize the errors of the previous tree. XGBoost employs regularization to prevent overfitting and improve model generalization. XGBoost is fast, accurate, and suitable for structured/tabular data tasks. Parallel computation is supported by the algorithm, and missing values are managed internally.

Lag Features

To capture autoregressive trends, the past 7 days of TMAX, TMIN, and PRCP were computed and appended as:

Method-1: 14-day lags for TMAX, TMIN, and PRCP. The dataset was split into training (90%) and validation (10%) subsets without shuffling to preserve temporal structure.

Method-2:

- TMAX_lag_1 to TMAX_lag_7
- TMIN_lag_1 to TMIN_lag_7
- PRCP_lag_1 to PRCP_lag_7

Rolling Statistics

Short-term trends were modeled using:

- 3-day average of TMAX (TMAX_3day_avg)
- 3-day sum of PRCP (PRCP_3day_sum)
- Daily TMAX trend (TMAX_trend) computed as the difference between consecutive days.

Records of missing values due to lag or rolling operations were removed.

Hyperparameter Tuning

XG Boost Regression-Used RandomizedSearch CV with parameters like:

- Method-1: 'n_estimators': [100, 200, 300], 'max_depth': [3, 5, 7], 'learning_rate': [0.01, 0.05, 0.1], 'subsample': [0.7, 0.8, 1.0], 'colsample_bytree': [0.7, 0.8, 1.0] and CV=3
- Method-2: 'n_estimators': [100, 200, 300], 'max_depth': [3, 5, 7], 'learning_rate': [0.01, 0.05, 0.1], 'subsample': [0.7, 0.9, 1.0], 'colsample_bytree': [0.7, 0.9, 1.0] and CV=3

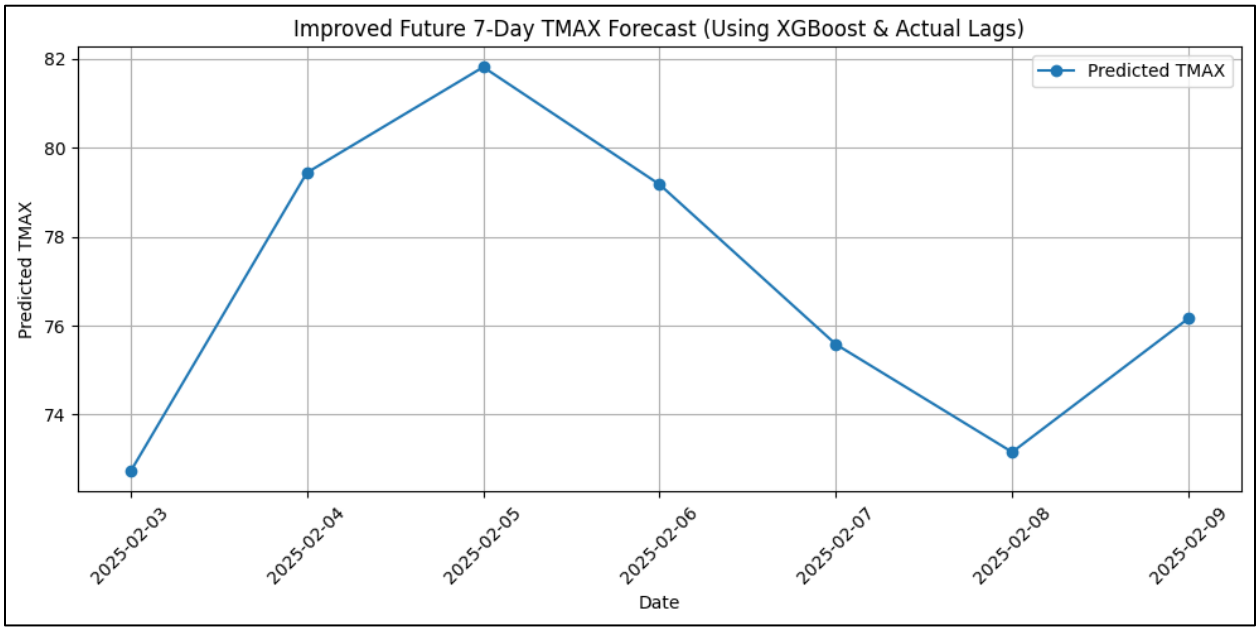
Method-1: Forecasting of TMAX Using Actual Lag Features from manual inputs

Lag features for the previous 14 days were created for TMAX, TMIN, and PRCP. Solar radiation and rolling statistics were excluded to evaluate model robustness under reduced feature complexity. A 7-day TMAX forecast was generated by manually inputting future TMIN and PRCP values.

```
🔍 Train Set Performance:
R²: 0.9157
MAE: 2.8928
MSE: 16.596
RMSE: 4.0738
```

```
🧪 Test Set Performance:
R²: 0.8844
MAE: 3.1445
MSE: 19.1981
RMSE: 4.3816
```

```
📈 Improved Future 7-Day TMAX Forecast (XGBoost):
        DATE  TMIN  PRCP  Predicted_TMAX
0 2025-02-03    45   0.0       72.721283
1 2025-02-04    65   0.0       79.442802
2 2025-02-05    68   0.0       81.819855
3 2025-02-06    67   0.2       79.177643
4 2025-02-07    54   0.0       75.579773
5 2025-02-08    50   0.0       73.153572
6 2025-02-09    52   0.0       76.163971
```



The forecast showed a smooth and consistent trend across the prediction horizon, with no abrupt fluctuations, highlighting the model's ability to maintain temporal continuity. The model correctly captures the initial rise in temperature, peaking around February 5, 2025, followed by a steady decline and slight rebound. Although few discrepancies exist in the magnitude of predicted values, particularly underestimations compared to the higher actual TMAX values, the model effectively tracks the turning points, showing improved responsiveness to temperature changes compared to previous models.

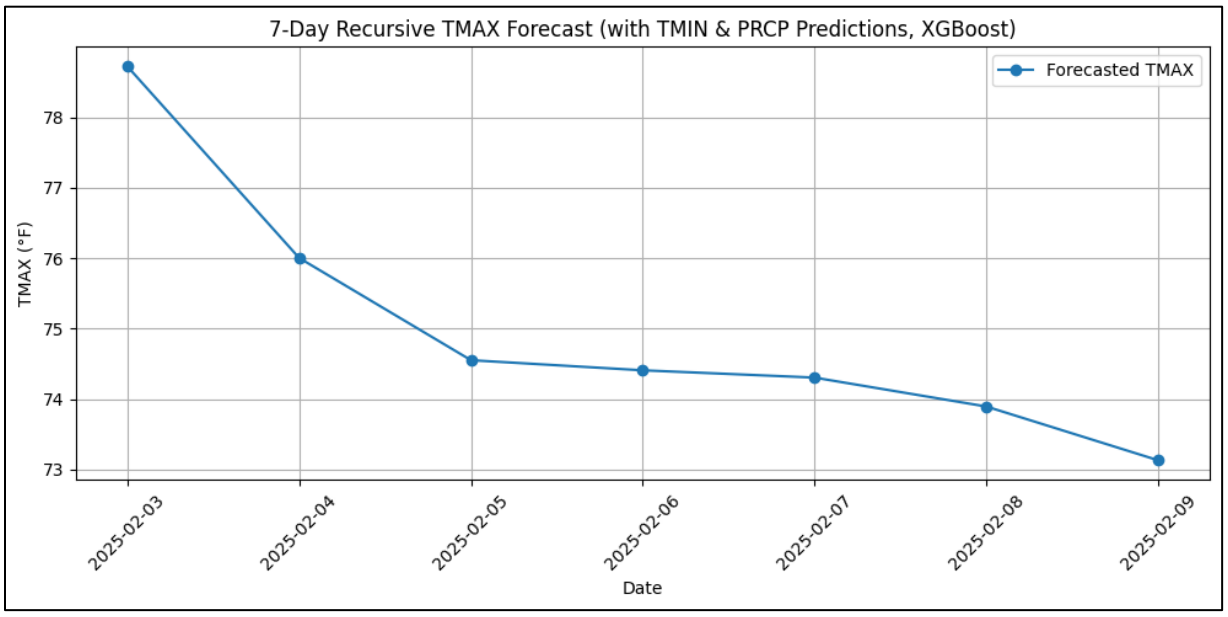
Method-2: Forecasting TMAX Using Recursive Multi-Stage Feature Engineering

A recursive forecasting approach was used, starting with the last seven days of historical data as the initial seed. TMIN and PRCP were predicted for Day 1 and, along with other features, used to forecast TMAX. Each day's predictions were appended to the dataset and used to forecast the next day's values. This iterative process reflects real-world forecasting, where each prediction depends on prior forecasts.

```
🔍 TMAX Model – Train Set Performance:
R²: 0.9991
MAE: 0.3146
MSE: 0.1719
RMSE: 0.4147
```

```
🧪 TMAX Model – Test Set Performance:
R²: 0.9969
MAE: 0.4683
MSE: 0.5261
RMSE: 0.7253
```

	DATE	Predicted_TMAX
0	2025-02-03	78.720047
1	2025-02-04	76.001839
2	2025-02-05	74.551331
3	2025-02-06	74.407867
4	2025-02-07	74.304459
5	2025-02-08	73.893852
6	2025-02-09	73.130913



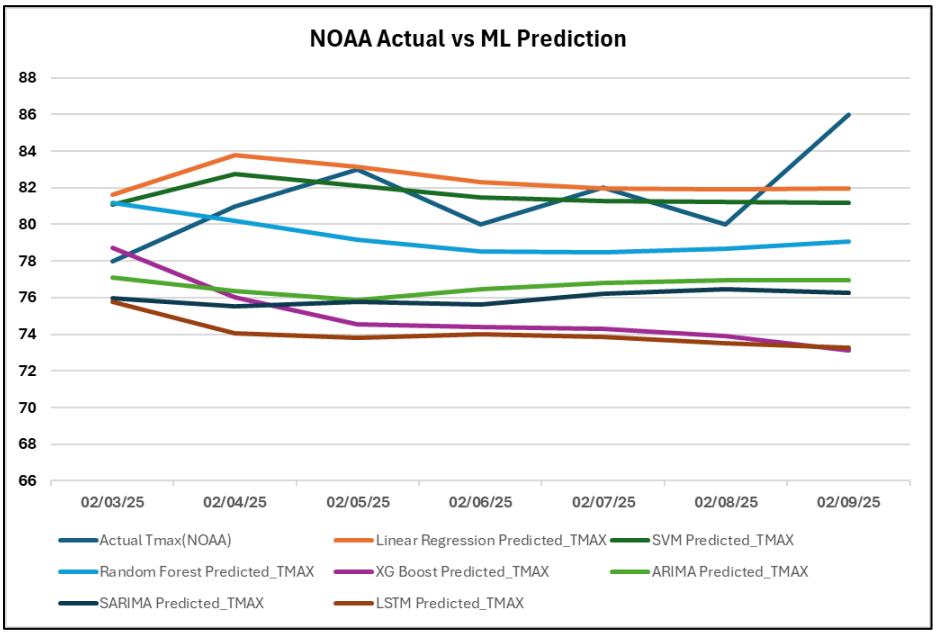
The XGBoost model achieved excellent performance on the training (R^2 = 0.9991, RMSE = 0.4147) and test sets (R^2 = 0.9969, RMSE = 0.7253), indicating strong fitting and generalization. However, the 7-day recursive TMAX forecast shows a consistent decline in predicted temperatures without capturing the typical fluctuations observed in actual NOAA data. Despite low error metrics, the model exhibits a smoothing effect, suggesting that while it predicts average trends well, it struggles with preserving the natural variability over multiple recursive steps.

Performance Evaluation

The ML models were compared to find out the best out of using the below metrics and forecast values.

Models	R-Squared	RMSE	MSE	MAE
Linear Regression	1	0	0	0
SVM	-0.274	14.76	217.93	10.17
Random Forest	0.9949	0.9117	0.8311	0.5098
XG Boost	0.9969	0.7253	0.5261	0.4683
ARIMA	-1.772	21.521	463.149	19.015
SARIMA	-1.655	21.063	443.649	18.581
LSTM		6.44		4.66

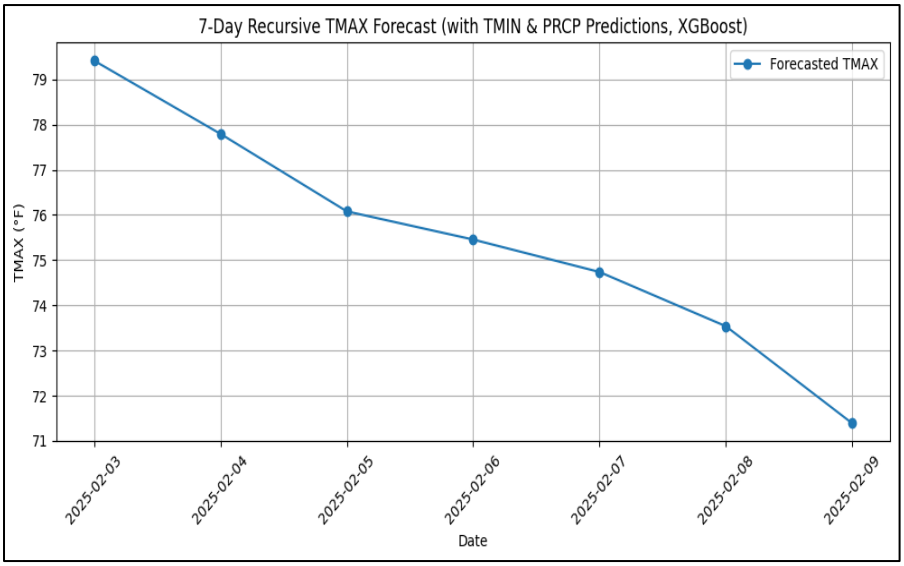
		USC00411048 - BRENHAM, TX US						
		Linear Regression	SVM	Random Forest	XG Boost	ARIMA	SARIMA	LSTM
DATE	Actual Tmax(NOAA)	Predicted_TMAX	Predicted_TMAX	Predicted_TMAX	Predicted_TMAX	Predicted_TMAX	Predicted_TMAX	Predicted_TMAX
02/03/25	78	81.60	81.07	81.16	78.72	77.09	75.96	75.77
02/04/25	81	83.76	82.75	80.17	76.00	76.35	75.52	74.05
02/05/25	83	83.14	82.14	79.15	74.55	75.87	75.77	73.81
02/06/25	80	82.33	81.49	78.53	74.41	76.48	75.65	74.01
02/07/25	82	81.97	81.29	78.47	74.30	76.82	76.22	73.85
02/08/25	80	81.92	81.21	78.67	73.89	76.94	76.46	73.51
02/09/25	86	81.96	81.20	79.05	73.13	76.95	76.28	73.26

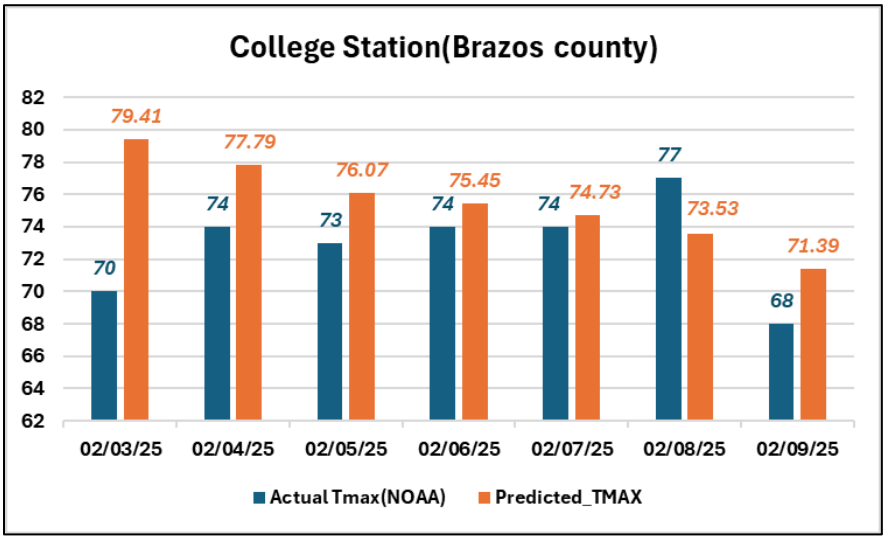


After comparing the performance of all models, XGBoost was selected as the best model for forecasting other stations. It achieved the highest R^2 (0.9969) and maintained low RMSE (0.7253) and MAE (0.4683), reflecting strong accuracy and generalization. Unlike models like ARIMA, SARIMA, and SVM, which either underfit or produced large errors, XGBoost effectively captured temperature trends and turning points. It also performed better than Random Forest by preserving more of the natural variability without excessive smoothing. Thus, XGBoost offers the most reliable foundation for extending predictions to additional locations. It also captured turning points better in your earlier plots compared to Random Forest and LSTM.

Prediction For Other Station

Based on performance across all metrics, XGBoost was selected for forecasting other stations due to its high accuracy, low error rates, and better ability to track temperature variations.





			XG Boost	
	DATE	Actual Tmax(NOAA)	Predicted_TMAX	Difference
1	02/03/25	70	79.41	-9.41
2	02/04/25	74	77.79	-3.79
3	02/05/25	73	76.07	-3.07
4	02/06/25	74	75.45	-1.45
5	02/07/25	74	74.73	-0.73
6	02/08/25	77	73.53	3.47
7	02/09/25	68	71.39	-3.39

Although XGBoost outperformed other models in our evaluations, its forecasting results at College Station Easterwood Field, TX, still show notable discrepancies when compared to the actual NOAA data. As seen in the graph, while the model captures the general trend, it struggles to match the exact temperature values, leading to consistent over- or under-predictions. Despite these limitations, XGBoost remains the best-performing model among those tested, offering the most reliable foundation for forecasting across different stations. Further model tuning and additional feature engineering could help improve future accuracy.

Limitations

One of the key challenges faced was sourcing accurate and complete Local Station weather data. Important variables such as humidity, wind speed, and pressure were unavailable, limiting the depth and precision of the forecasting models. Handling missing values also posed difficulties, as imputation methods may have reduced model accuracy, particularly for extreme temperature values. Traditional models like ARIMA and SARIMA significantly underperformed, failing to capture the complexity of the data. Although deep learning models like LSTM offered potential improvements, they required high computational resources, making them less practical for the current setup.

Future Recommendations

Enhance Input Features

Add key weather variables like humidity, wind speed, and pressure to enrich model inputs. Incorporate historical climate patterns and solar radiation for better temporal coverage. Utilize real-time sensor data where available to improve the timeliness and granularity of inputs.

Strengthening Model Architecture

Develop hybrid models combining machine learning and deep learning approaches. Explore advanced architectures like Temporal Convolutional Networks or attention mechanisms. Fine-tune sequence models (e.g., LSTM, GRU) with hyperparameter optimization and regularization.

Improve Generalization

Train models across diverse climate zones to improve adaptability. Leverage transfer learning by adapting pre-trained models between stations. Apply time series-specific cross-validation methods like rolling-origin evaluation.

Deployment & Scale

Extend forecast horizons by capturing seasonal and cyclic patterns. Set up cloud-based pipelines for large-scale, multi-station deployment. Enable automatic retraining using incoming real-time data to maintain accuracy.

Boost Transparency
Use SHAP and LIME to explain and interpret model predictions. Generate visual insights into feature importance and temporal behavior. Integrate interpretability metrics to validate and communicate model reliability.

AI-driven Weather Forecasting Models

We reviewed GraphCast (artificial intelligence weather forecasting model), Meteomatics (API weather data provider), and IBM Environmental Intelligence Suite (EIS) to test whether they are the best fit for GaiaDhi, an agribusiness company seeking to leverage the emerging forecasting technologies in a bid to enhance operation efficiency, especially in farm decision-making.

GraphCast AI Model
GraphCast operates a Graphical Neural Network that was previously trained from historical weather patterns to produce highly accurate 10-day forecasts using deep learning techniques. GraphCast, however, is high-hardware-infrastructure-and-computing-power-requiring, which makes it costly and infeasible for a firm like GaiaDhi. Additionally, training the AI model and processing large datasets are extra complexities to the software. While its precision and high forecasting powers, the heavy cost and infrastructure needs make it less practical for GaiaDhi now.

Meteomatics API
Meteomatics offers weather data of high resolution through an API, which can be readily incorporated into existing systems. It provides localized forecasting, which is useful for farming operations specifically, allowing GaiaDhi to make more precise, data-driven choices. One of its biggest advantages is that it does not require significant hardware or computer infrastructure like AI models. Further, the API-based design enables easier deployment and faster implementation times. However, high-end access to precise weather data sets comes at a very steep price. Additionally, although integration technically becomes less complicated than in AI models, it still requires knowledge of correctly interpreting the data and translating it into functional goals. In general, Meteomatics is a more comfortable choice than GraphCast, with the ideal ratio of forecast quality and simplicity of integration, although cost and reliance on third-party data providers remain significant issues. Another limitation is that we can observe values only for future forecasts and not prior month forecasts, and we were not able to set the query to observe map coordinates for our location Brenham, TX.

IBM Environmental Intelligence Suite
IBM EIS provides RESTful APIs for seamless integration into existing systems and provides 15-day forecasts with detailed analytics. It leverages machine learning and AI to analyze weather impacts on operations and provides advanced forecasting capabilities. EIS is an enterprise solution, and as such, it might have high subscription and customization charges. It also relies on third-party

data providers like The Weather Company. Even with its moderate to high chance of integration with existing systems, making it a probable choice, GaiaDhi needs to carefully track related costs and dependencies. But we could not adequately test its features using API simulation in this review.

Feature	GraphCast (AI Model)	Meteomatics (Weather API)	IBM Environmental Intelligence Suite
Forecasting Method	Uses a Graphical Neural Network trained on historical weather data	Provides high-resolution weather data via an API	Provides RESTful APIs for seamless integration with existing systems
Benefits	Highly accurate 10-day forecasts Helps optimize farming operations	Localized and high-resolution forecasts Easier to integrate than AI models	Delivers 15-day forecasts with granular insights, Integrates AI, and machine learning to analyze weather impacts on operations
Challenges	High hardware & computing power requirements Expensive AI model training and data processing	Premium access to precise weather data is costly Requires technical expertise to interpret data	IBM EIS is a premium solution with a potentially high subscription and customization cost, EIS relies on third-party data providers like The Weather Company
Feasibility for GaiaDhi	Less feasible due to high infrastructure and integration costs	More feasible, but cost of premium data and reliance on external sources are concerns	Moderate to high feasibility to integrate with existing sources

Conclusion

Although all three alternatives offer some valuable forecasting capability, the expense, hardware cost, and integration problems make them less appropriate for GaiaDhi in its current form. Among the options, Meteomatics appears to be the most scalable and practical short-term solution to implement with trial period, offering a nice usability-functionality balance. IBM Environmental Intelligence Suite also has a compelling argument based on quality of insights and integration potential, but budget implications and reliance on third-party sources of information need to be weighed carefully. Lastly, careful cost-benefit analysis and staged implementation will be the key to optimizing the application of weather forecasting in GaiaDhi.

References

- https://www.noaa.gov/
- https://power.larc.nasa.gov/api/temporal/daily/point?parameters=PRECTOTCORR&community=ag&longitude={longitude}&latitude={latitude}&start={start}&end={end}&format=JSON
- https://www.meteomatics.com/en/sign-up-weather-api-test-account/
- https://www.ibm.com/docs/en/environmental-intel-suite?topic=reference-weather-data-apis
- https://gis-txdot.opendata.arcgis.com/datasets/texas-county-boundaries-line/explore?location=30.398901%2C-95.904804%2C9.22
- https://www2.census.gov/geo/tiger/TIGER2024/COUNTY/